

Arduino Microcontroller Concepts (CSE 4820)

CHAPTER 2 - Arduino Programming



Program: B. TECH., 7th Semester

SIKSHA 'O' ANUSHANDHAN (Deemed to be) UNIVERSITY

Arduino - Data Types

void	Boolean	char	Unsigned char	byte	int	Unsigned int	word
long	Unsigned long	short	float	double	array	String-char array	String-object

void

- ✓ used only in function declarations
- ✓ return no information to the function

```
void loop ( ) {
```

```
// rest of the code
```

```
}
```

Boolean

- ✓ Holds one of two values, true or false
- ✓ Each Boolean variable occupies one byte of memory

```
Boolean Val = false ; // declaration of variable with type  
Boolean and initialize it with false
```

```
Boolean state = true ; // declaration of variable with type  
Boolean and initialize it with true
```

Arduino - Data Types contd.

Char

- ✓ takes up one byte of memory
- ✓ written in single quotes : 'A'
- ✓ A signed char is same as an ordinary char and has a range from -128 to +127
- ✓ multiple characters, strings use double quotes: "ABC"
- ✓ characters are stored as numbers (ASCII chart)
 - 'A' + 1 has the value 66,
 - since the ASCII value of the capital letter A is 65.

```
char chr_a = 'a' ; //declaration of variable with  
type char and initialize it with character a  
char chr_c = 97 ; //declaration of variable with  
type char and initialize it with character 97
```

ASCII Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL 0	SOH 1	STX 2	ETX 3	EOT 4	ENQ 5	ACK 6	BEL 7	BS 8	HT 9	LF 10	VT 11	FF 12	CR 13	SO 14	SI 15
1	DLE 16	DC1 17	DC2 18	DC3 19	DC4 20	NAK 21	SYN 22	ETB 23	CAN 24	EM 25	SUB 26	ESC 27	FS 28	GS 29	RS 30	US 31
2	SPC 32	! 33	" 34	# 35	\$ 36	% 37	& 38	' 39	(40	) 41	* 42	+ 43	, 44	- 45	. 46	/ 47
3	0 48	1 49	2 50	3 51	4 52	5 53	6 54	7 55	8 56	9 57	: 58	; 59	< 60	= 61	> 62	? 63
4	@ 64	A 65	B 66	C 67	D 68	E 69	F 70	G 71	H 72	I 73	J 74	K 75	L 76	M 77	N 78	O 79
5	P 80	Q 81	R 82	S 83	T 84	U 85	V 86	W 87	X 88	Y 89	Z 90	[91	\ 92	] 93	^ 94	_ 95
6	` 96	a 97	b 98	c 99	d 100	e 101	f 102	g 103	h 104	i 105	j 106	k 107	l 108	m 109	n 110	o 111
7	p 112	q 113	r 114	s 115	t 116	u 117	v 118	w 119	x 120	y 121	z 122	{ 123	 124	} 125	~ 126	DEL 127

unsigned char

- ✓ unsigned data type occupies one byte of memory
- ✓ encodes numbers from 0 to 255

```
unsigned char chr_y = 121 ; // declaration of variable with type Unsigned char and initialize it with character y
```

Arduino - Data Types contd.

Byte

- ✓ stores an 8-bit unsigned number, from 0 to 255

`byte m = 25 ;` //declaration of variable with type byte and initialize it with 25

int

- ✓ primary data-type for number storage
- ✓ stores a 16-bit (2-byte) value
- ✓ yields a range of -32,768 to 32,767 (minimum value of -2^{15} and a maximum value of $(2^{15}) - 1$).

`int counter = 32 ;` // declaration of variable with type int and initialize it with 32

Unsigned int

- ✓ only store positive values
- ✓ store a 2 byte value
- ✓ range of 0 to 65,535 (2^{16} - 1)

`unsigned int counter = 60 ;` // declaration of variable with type unsigned int and initialize it with 60

Word

- ✓ stores a 16-bit unsigned number

`word w = 1000 ;` //declaration of variable with type word and initialize it with 1000

Arduino - Data Types contd.

Long

- ✓ extended size variables for number storage
- ✓ store 32 bits (4 bytes), from -2,147,483,648 to 2,147,483,647.

`long velocity = 102346 ;` //declaration of variable with type Long and initialize it with 102346

unsigned long

- ✓ extended size variables for number storage
- ✓ not store negative numbers
- ✓ store 32 bits (4 bytes), from 0 to 4,294,967,295 ($2^{32} - 1$).

`unsigned Long velocity = 101006 ;` // declaration of variable with type Unsigned Long and initialize it with 101006

short

- ✓ stores a 16-bit (2-byte) value
- ✓ range of -32,768 to 32,767 (minimum value of -2^{15} and a maximum value of $(2^{15}) - 1$).

`short val = 13 ;` //declaration of variable with type short and initialize it with 13

float

- ✓ number has a decimal point
- ✓ used to approximate the analog and continuous values because they have greater resolution than integers.
- ✓ can be as large as $3.4028235E+38$ and as low as $-3.4028235E+38$.
- ✓ stored as 32 bits (4 bytes) of information

`float num = 1.352;` //declaration of variable with type float and initialize it with 1.352

double

- ✓ Double precision floating-point number occupies four bytes

`double num = 45.352 ;` // declaration of variable with type double and initialize it with 45.352

Important Arduino Data Types

int

whole numbers

```
int count = 10;
```

float

decimal values

```
float t = 25.5;
```

bool

true / false

```
bool on = true;
```

char

single character

```
char c = 'A';
```

byte

0 to 255

```
byte n = 200;
```

String

text

```
String s = "Hi";
```

Arduino data types for 8-bit boards such as the Uno

Arduino data types for 8-bit boards such as the Uno

Numeric types	Bytes	Range	Use
int	2	−32768 to 32767	Represents positive and negative integer values.
unsigned int	2	0 to 65535	Represents only positive values; otherwise, similar to int.
long	4	−2147483648 to 2147483647	Represents a very large range of positive and negative values.
unsigned long	4	0 to 4294967295	Represents a very large range of positive values.
float	4	3.4028235E+38 to −3.4028235E+38	Represents numbers with fractions; use to approximate real-world measurements.
double	4	Same as float	In Arduino, double is just another name for float.
bool	1	false (0) or true (1)	Represents true and false values.
char	1	−128 to 127	Represents a single character. Can also represent a signed numeric value between −128 and 127.
byte	1	0 to 255	Similar to char, but for unsigned values.

Other types Use

String	Represents a sequence of characters typically used to contain text.
void	Used only in function declarations where no value is returned.

int: 2 bytes, −32768 to 32767

unsigned int: 0 to 65535

long: 4 bytes, large integer range

float: decimal/fractional values

bool: false or true

char: single character

String: sequence of characters

Arduino - Variables & Constants

What is Variable Scope?

A scope is a region of the program and there are **three** places where variables can be declared.

- Inside a function or a block, which is called **local variables**.
- In the definition of function parameters, which is called **formal parameters**.
- Outside of all functions, which is called **global variables**.

Local Variables

- ✓ declared inside a function or block
- ✓ used only by the statements that are inside that function or block of code.

```
void setup () {
```

```
}
```

```
void loop () {
```

```
  int x , y ;
```

```
  int z ; local variable declaration
```

```
  x = 0;
```

```
  y = 0; actual initialization
```

```
  z = 10;
```

```
}
```

Arduino - Variables & Constants contd.

Global Variables

- ✓ defined outside of all the functions, usually at the top of the program.
- ✓ hold their value throughout the life-time of your program.
- ✓ accessed by any function.
- ✓ available for use throughout your entire program after its declaration.

```
int T , S ;  
float c = 0 ; Global variable declaration
```

```
void setup () {  
  
}
```

```
void loop () {  
  int x , y ;  
  int z ; Local variable declaration  
  x = 0;  
  y = 0; actual initialization  
  z = 10;  
}
```

Arduino - Functions

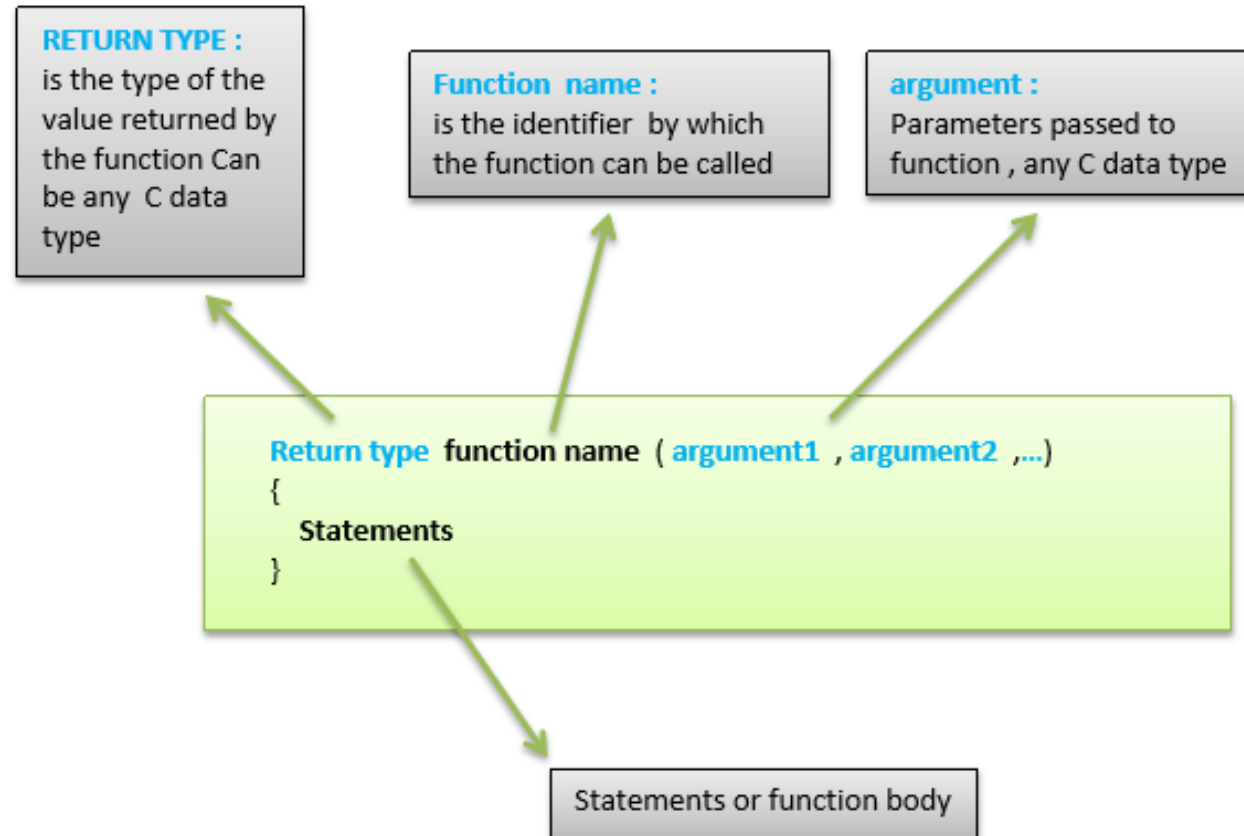
A function is a block of code that performs a specific task. It can be called whenever that task needs to be performed.

Advantages of Using Function :-

- ✓ Code Reusability
- ✓ Easy Debugging
- ✓ Reduces Code Length
- ✓ Easy to Understand
- ✓ Easy Maintenance

There are two required functions in an Arduino sketch or a program i.e. **setup ()** and **loop()**. Other functions must be created outside the brackets of these two functions.

Common syntax to define a function



Function Declaration

A function declaration is used to tell the compiler about a function before it is used or called.

- Function return type
- Function name
- Function argument type, **here you must add the argument name**
- The function body (statements inside the function executing when the function is called)

```
int sum_func (int x, int y)    // function declaration
{
    int z = 0;
    z = x+y ;
    return z;                  // return the value
}
```

```
void setup () {
    Statements                  // group of statements
}
```

```
void loop () {
    int result = 0 ;
    result = Sum_func (5,6) ;    // function call
}
```

Programming

Program Structure

```
//Globle variable
...

void setup()
{
  //Initialization
}

void loop()
{
  //Main Program
}
```

Example 2

You need to use the following:

Which Pin (0, 1, 3, ...) are you using?

pinMode(pin, mode);

A Digital Pin can either be an INPUT or an OUTPUT. Since we shall use it to turn-on a LED, we set it to OUTPUT.

digitalWrite(pin, value);

Turn-on
LED

Turn-off
LED

A Digital Pin can have 2 values, either HIGH or LOW

delay(ms);

The delay() function makes a small pause in milliseconds (ms), e.g., delay(1000) pause the program for 1 second

Arduino Language Reference: <https://www.arduino.cc/en/Reference>

Arduino Program

Example 2

```
void setup()
{
    pinMode(8, OUTPUT);
}

void loop()
{
    digitalWrite(8, HIGH);    // Turn on the LED
    delay(1000);              // Wait for one second
    digitalWrite(8, LOW);     // Turn off the LED
    delay(1000);              // Wait for one second
}
```

TRY IT OUT!

Arduino Program

Example 2

```
int ledPin = 8;
void setup()
{
    pinMode(ledPin, OUTPUT);
}

void loop()
{
    digitalWrite(ledPin, HIGH);    // Turn on the LED
    delay(1000);                  // Wait for one second
    digitalWrite(ledPin, LOW);     // Turn off the LED
    delay(1000);                  // Wait for one second
}
```

En ørliten forbedring. Vi bruker en variabel til å definere pinne-nummeret

TRY IT OUT!

Creating and Using Functions

```
int z;  
  
void setup()  
{  
  }  
  
void loop()  
{  
  z = calculate(2,3);  
}  
  
float calculate(int x, int y)  
{  
  return (x + y);  
}
```

Using the Function

Creating the Function

TRY IT OUT!

Here are some Arduino Examples you should try.

Make sure your Arduino is connected to the PC and start the Code Editor



"Hello World" Example

TRY IT OUT!

Create the following program:

Open the "Serial Monitor" in order to see the output

```
void setup()
{
  Serial.begin(9600);

  Serial.println("Hello, world!");
}

void loop()
{
}
```

"Hello World" Example

TRY IT OUT!

Create the following program:

Open the "Serial Monitor" in order to see the output

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  Serial.println("Hello, world!");
  delay(1000);
}
```

Example

Create the following program:

Open the “Serial Monitor” in order to see the output

```
int z;int a;int b;
void setup()
{
    Serial.begin(9600);
}
void loop()
{
    a = random(100);
    b = random(100);
    z = calculate(a,b); //Adding 2 Numbers

    //Write Values to Serial Monitor
    Serial.print(a);
    Serial.print(" + ");
    Serial.print(b);
    Serial.print(" = ");
    Serial.println(z);

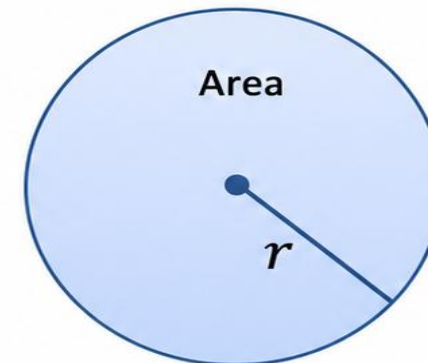
    delay(1000);
}
float calculate(int x, int y)
{
    return (x + y);
}
```

TRY IT OUT!

Creating Functions

Create a function that calculates the area of a circle with a given radius.

Write the result to the Serial Monitor.



TRY IT OUT!

Solution

```
void setup()
{
    float area;

    Serial.begin(9600);
    // calculate the area of a circle with radius of 9.2
    float r=9.2;
    area = CircleArea(r);
    Serial.print("Area of circle is: ");
    // print area to 4 decimal places
    Serial.println(area, 4);
}

void loop()
{
}

// calculate the area of a circle
float CircleArea(float radius)
{
    float result;
    const float pi = 3.14;

    result = pi * radius * radius;

    return result;
}
```

TRY IT OUT!

Arduino - Time

Arduino provides **four** different time manipulation functions

1. delay () function

- ✓ Accepts a single integer (or number) argument
- ✓ This number represents the time (measured in milliseconds).

2. delayMicroseconds () function

- ✓ Accepts a single integer (or number) argument.
- ✓ There are a thousand microseconds in a millisecond, and a million microseconds in a second.

3. millis () function

- ✓ Used to return the number of milliseconds at the time, the Arduino board begins running the current program.

4. micros () function

- ✓ Returns the number of microseconds from the time, the Arduino board begins running the current program.
- ✓ This number overflows i.e. goes back to zero after approximately 70 minutes.

Arduino - delay () function

Syntax

delay (ms) ; where, ms is the time in milliseconds to pause (unsigned long).

The program should wait until moving on to the next line of code when it encounters this function. However, the problem is, the delay() function is not a good way to make your program wait, because it is known as a “**blocking**” function.

Objective : Turns on and off a light emitting diode(LED) connected to a digital pin, in intervals of 2 seconds.

```
int ledPin = 13;                // LED connected to digital pin 13

void setup() {
  pinMode(ledPin, OUTPUT);      // sets the digital pin as output
}

void loop() {
  digitalWrite(ledPin, HIGH);   // sets the LED on
  delay(1000);                  // waits for a second
  digitalWrite(ledPin, LOW);    // sets the LED off
  delay(1000);                  // waits for a second
}
```

Arduino - delayMicroseconds () function

Syntax

`delayMicroseconds (us) ;` where, `us` is the number of microseconds to pause (unsigned int)

Objective : Turns on and off a light emitting diode(LED) connected to a digital pin, in intervals of **2 milli seconds**.

```
int ledPin = 13;                // LED connected to digital pin 13

void setup() {
  pinMode(ledPin, OUTPUT);      // sets the digital pin as output
}

void loop() {
  digitalWrite(ledPin, HIGH);    // sets the LED on
  delayMicroseconds(1000);       // waits for a second
  digitalWrite(ledPin, LOW);     // sets the LED off
  delayMicroseconds(1000);       // waits for a second
}
```

For delays longer than a few thousand microseconds, you should use the `delay()` function instead.

Arduino - millis () function

Syntax

```
millis ();
```

- ✓ This function returns milliseconds from the start of the program.
- ✓ This number overflows i.e. goes back to zero after approximately 50 days.

```
unsigned long time;
```

```
void setup() {  
  Serial.begin(9600);  
}
```

```
void loop() {  
  Serial.println("Time:");  
  time = millis();    //prints time since program started  
  Serial.println(time);    // wait a second so as not to send massive amounts of data  
  delay(1000);  
}
```

Arduino - micros () function

Syntax

micros ();

This function returns number of microseconds since the program started (unsigned long)

- ✓ The micros() function returns the number of microseconds from the time, the Arduino board begins running the current program. This number overflows i.e. goes back to zero after approximately 70 minutes.
- ✓ On 16 MHz Arduino boards (e.g. Nano), this function has a resolution of four microseconds (i.e. the value returned is always a multiple of four).

```
unsigned long time;  
void setup() {  
  Serial.begin(9600);  
}
```

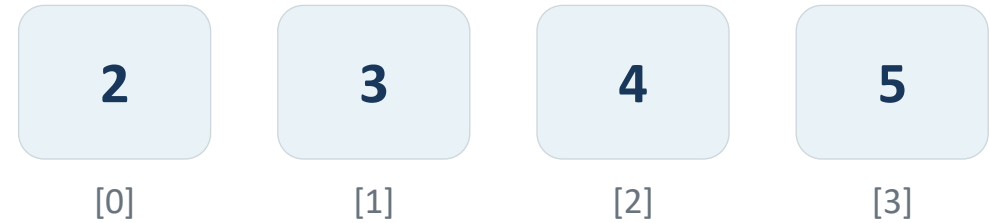
```
void loop() {  
  Serial.print("Time:");  
  time = micros();  
  Serial.println(time);  
  delay(1000);  
}
```

//prints time since program started

// wait a second so as not to send massive amounts of data

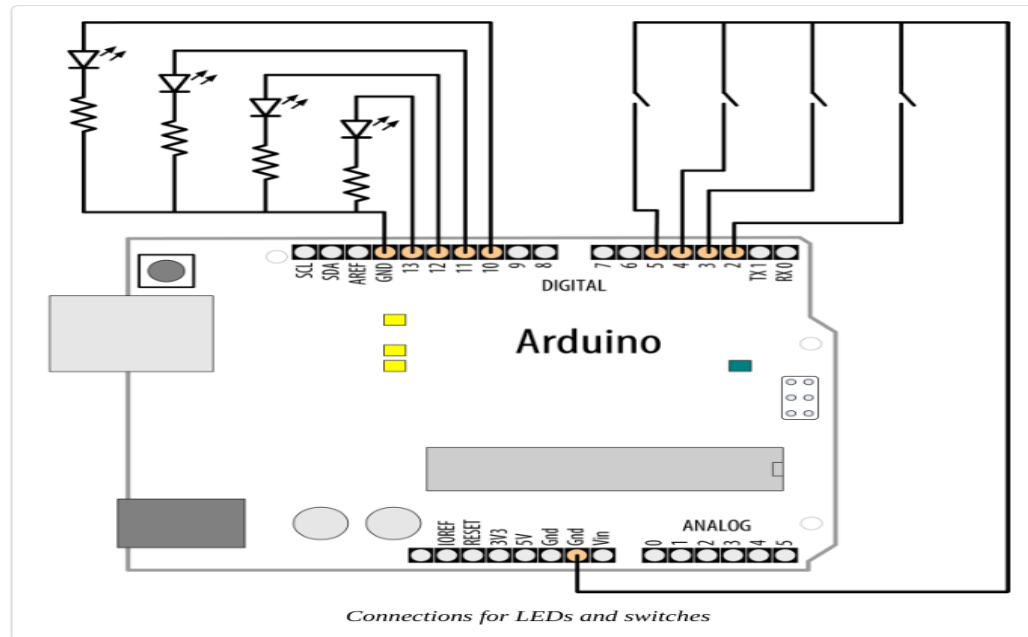
Arrays: A Group of Values

- ✓ An array stores multiple values of the same type.
- ✓ Elements are accessed using an index.
- ✓ Arduino/C/C++ arrays start at index 0.
- ✓ A four-element array has indexes 0, 1, 2 and 3.



```
int inputPins[] = {2, 3, 4, 5};
```

Arrays in an Arduino Project



Discussion

Arrays are collections of consecutive variables of the same type. Each variable in the collection is called an *element*. The number of elements is called the *size* of the array.

The Solution demonstrates a common use of arrays in Arduino code: storing a collection of pins. Here the pins connect to switches and LEDs (a

Arrays can store collections of input and output pins.

Example: switches $\rightarrow$ inputPins[]

Example: LEDs → `ledPins[]`

A for loop can process all elements efficiently.

```
for (int index = 0; index < 4; index++)
{
    pinMode(ledPins[index], OUTPUT);
}
```

Arrays

Here we shall use arrays in the Arduino program.

Create this program from scratch and open the Serial Monitor to see the result.

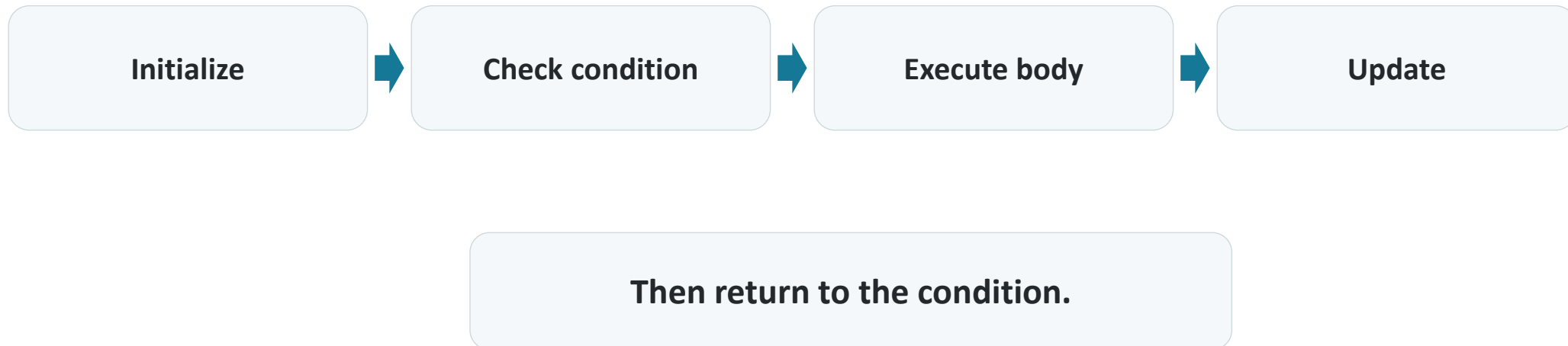
TRY IT OUT!

```
const int arraysize = 100;
int x;
int sum = 0;
float average = 0;
int myarray[arraysize];

void setup()
{
    Serial.begin(9600);
}
void loop()
{
    sum = 0;
    for (int i = 0; i < arraysize; i++)
    {
        x = random(200);
        myarray[i] = x;
    }
    sum = calculateSum(myarray);
    average = sum / 100;
    Serial.print(" Sum = ");
    Serial.println(sum);
    Serial.print(" , Average = ");
    Serial.println(average);
    delay(1000);
}
int calculateSum (int sumarray[])
{
    for (int i = 0; i < arraysize; i++)
    {
        sum = sum + sumarray[i];
    }
    return sum;
}
```

The for Loop

```
for (initialization; condition; update)
{
    // repeated statements
}
```



2.14 Repeating Statements with a Counter

Problem

You want to repeat one or more statements a certain number of times. The for loop is similar to the while loop, but you have more control over the starting and ending conditions.

Solution

This sketch counts from zero to three by printing the value of the variable `i` in a for loop:

```
/*  
  ForLoop sketch  
  demonstrates for loop  
*/  
  
void setup() {  
  Serial.begin(9600);  
}  
  
void loop(){  
  Serial.println("for(int i=0; i < 4; i++)");  
  for(int i=0; i < 4; i++)  
  {  
    Serial.println(i);  
  }  
  delay(1000);  
}
```

The Serial Monitor output from this is as follows (it will be displayed over and over):

```
for(int i=0; i < 4; i++)  
0  
1  
2  
3
```

For Loop

In this program we use a For Loop to find the Sum of 100 Random Numbers.

Then we find the Average.

The Sum and Average should be written to the Serial Monitor.

```
int x;
int sum = 0;
float average = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  sum = 0;
  for (int i = 0; i < 100; i++) {
    x = random(100);
    sum = sum + x;
  }
  average = sum / 100.0; // Ensure floating point
division
  Serial.print("Sum = ");
  Serial.println(sum);
  Serial.print(" Average = ");
  Serial.println(average);
  delay(1000);
}
```

String: Working with Text

- ✓ Arduino provides a String object for storing and manipulating text.
- ✓ String uses an uppercase S.
- ✓ Useful methods include `length()`, `concat()`, `indexOf()`, `substring()`.
- ✓ Strings are indexed from 0, like arrays.

```
String text1 = "This text";  
String text2 = " has more characters";  
  
text1.concat(text2);  
Serial.println(text1);
```

Output: This text has more characters

Useful Arduino String Functions

length()

number of characters

concat()

combine strings

indexOf()

find text/character

substring()

extract part of a string

replace()

replace text

toInt() / toFloat()

convert text to number

Arduino String Functionality

This recipe demonstrates how to use Arduino Strings.

Load the following sketch onto your board, and open the Serial Monitor to view the results:

```
/*
  Basic_Strings sketch
*/

String text1 = "This text";
String text2 = " has more characters";
String text3; // to be assigned within the sketch

void setup()
{
  Serial.begin(9600);
  while(!Serial); // Wait for serial port (Leonardo, 32-bit boards)

  Serial.print("text1 is ");
  Serial.print(text1.length());
  Serial.println(" characters long.");

  Serial.print("text2 is ");
  Serial.print(text2.length());
  Serial.println(" characters long.");

  text1.concat(text2);
  Serial.println("text1 now contains: ");
  Serial.println(text1);
}
```

o/p- This text has more characters and more

```
void loop()
{
}
```

WHY NOT SERIAL?

For the Arduino Uno and most 8-bit boards, when you open the Serial Monitor in the Arduino IDE, it resets the board, which means that you will see any serial output that is generated in the setup function shortly after you open the Serial Monitor. However, on the Leonardo, and on SAMD-based boards, opening the serial port does not automatically reset the board, meaning you won't be able to open the Serial Monitor quick enough to capture the output. For this reason, you will see the `while(!Serial);` line in several setup functions in this chapter and throughout the book. See “[Serial Hardware Behavior](#)” for more details.

Discussion

This sketch creates three variables of type `String`, called `text1`, `text2`, and `text3`. Variables of type `String` have built-in capabilities for manipulating text. The statement `text1.length()` returns (provides the value of) the length (number of characters) in the string `text1`.

`text1.concat(text2)` combines the contents of strings; in this case, it appends the contents of `text2` to the end of `text1` (`concat` is short for *concatenate*).

The Serial Monitor will display the following:

```
text1 is 9 characters long.
text2 is 20 characters long.
text1 now contains:
This text has more characters
```

Another way to combine strings is to use the string addition operator. Add these two lines to the end of the setup code:

```
text3 = text1 + " and more";
Serial.println(text3);
```

Converting a Number to a String

- Useful when a numeric value must be displayed as text.
- Arduino String can automatically convert numeric values.
- `concat()` and `+` can append numbers to text.

```
int value = 127;  
String readout = "The reading was ";  
readout.concat(value);  
  
// or  
readout += value;
```

Result: The reading was 127

2.8 Converting a Number to a String

Problem

You need to convert a number to a string, perhaps to show the number on an LCD or other display.

Solution

The `String` variable will convert numbers to strings of characters. You can use literal values, or the contents of a variable. For example, the following code will work:

```
String myNumber = String(1234);
```

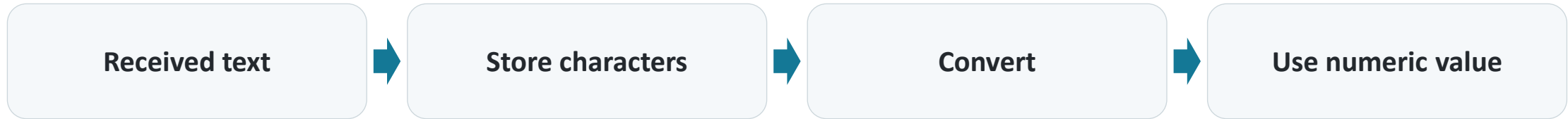
As will this:

```
int value = 127;  
String myReadout = "The reading was ";  
myReadout.concat(value);
```

Or this:

```
int value = 127;  
String myReadout = "The reading was ";  
myReadout += value;
```

Converting a String to a Number



```
char str[] = "250";  
int delayTime;  
  
delayTime = atoi(str);
```

atoi() → ASCII to int

Read an integer number from the Serial Monitor.

Serial.parseInt() is used in Arduino to read an integer number entered by the user through the Serial Monitor.

Example:-

```
int num;
```

```
void setup()  
{  
  Serial.begin(9600);  
}
```

```
void loop()  
{  
  num = Serial.parseInt();  
  Serial.println(num);  
}
```

If you type:

25

in the Serial Monitor, Serial.parseInt() reads 25 and stores it in num.